ChatGPT

مواصفات متطلبات النظام – DevPulse API

English Term	Arabic Term
API	واجهة برمجية
CRUD	إنشاء/قراءة/تحديث/حذف
Endpoint	نقطة نهاية
Actor	ممثل
Use Case	حالة استخدام
Entity	كيان
Component Diagram	مخطط المكونات
Deployment Diagram	مخطط النشر
ER Diagram	مخطط العلاقة الكيانية
Class Diagram	مخطط الأصناف
Sequence Diagram	مخطط التسلسل
Activity Diagram	مخطط النشاط
Authentication	مصادقة
Authorization	تفويض
Policy	سياسات التحكم بالوصول
Middleware	وسيط برمجي
Form Request	طلب نموذج (للتحقق)
Rate Limiting	تحديد معدل الطلبات
Data Dictionary	قاموس البيانات
Endpoint Spec	مواصفات نقطة النهاية
Glossary	جدول المصطلحات

الملخص التنفيذي

هذا المستند يحدد **متطلبات النظام** لنظام **DevPulse API**، وهو واجهة برمجية (RESTful API) مبنية على إطار عمل **Laravel**. يهدف النظام إلى دعم منصة محتوى (مدونة ومقالات تقنية) تخدم الواجهة الأمامية (تطبيق ويب Next.js وتطبيق جوال Flutter). يتيح النظام للمستخدمين تسجيل حساباتهم والتفاعل مع المحتوى (إنشاء وتعديل المقالات، إضافة التعليقات، إدارة الوسوم والتصنيفات)، كما يتيح للمشرفين (Admins) إدارة المحتوى بالكامل (إدارة التصنيفات والوسوم، وحذف أي محتوى مسيء).

المواصفات التالية تغطي المخططات والهياكل الضرورية لفهم النظام: مخطط سياق النظام (Context Diagram) يوضح حدود النظام والعلاقات الخارجية، **مخططات حالات الاستخدام** (Use Case) التي ترسم تفاعل الممثلين (الزائر، المستخدم، المشرف)، مخطط العلاقة الكيانية (ERD) لتصميم قاعدة البيانات، مخطط الأصناف (Class Diagram) لعرض الكيانات الرئيسية وعلاقاتها البرمجية، مخططات التسلسل (Sequence Diagrams) للتدفقات الأساسية (تسجيل المستخدم، تسجيل الدخول، إنشاء مقال، إضافة تعليق)، مخطط النشاط (Activity Diagram) لعملية إنشاء مقال، مخطط المكونات (Component Diagram) يوضح الوحدات الرئيسية للنظام، ومخطط النشر (Deployment Diagram) لبنية النشر. كما يتضمن المستند **جدول CRUD** للإجراءات الأساسية لكل كيان، **قاموس قاعدة البيانات** (جداول الحقول بأنواعها وقيودها)، ومخطط تفصيلي لنقاط النهاية (API Endpoints) مع الأساليب والطلبات والاستجابات المتوقعة.

يتضمن المستند أيضًا سجل قرارات التصميم (Architecture Decision Records) لمبررات اختيار بعض التقنيات (مثلاً: استخدام Laravel Sanctum للمصادقة عديمة الحالة)، ومخطط تنفيذ وجدول زمني مقترح (Implementation Roadmap) يُظهر مراحل التطوير المتوقعة لإنجاز نسخة MVP خلال 4-6 أسابيع (تم افتراضها لعدم وجود إطار زمني محدد في الوثيقة).

تم استخدام مبادئ **RESTful API** والأمن الأفضل (مثل التحقق من الصحة، تحديد معدل الطلبات، وحماية الـ CORS) لضمان جودة وسلامة النظام. جميع الرسوم التوضيحية بالإنجليزية (وفق لغة النمذجة UML)، بينما الشرح باللغة العربية. أي افتراض تم استخدامه في حال غياب تفاصيل محددة تم تمييزه بوضوح.

المحتويات

1. مقدمة
2. الرؤية العامة للنظام (مخطط السياق)
3. مخطط حالات الاستخدام
4. مخطط العلاقة الكيانية (ERD)
5. مخطط الأصناف (Class Diagram)
6. مخططات التسلسل
7. تسجيل مستخدم جديد
8. تسجيل الدخول
9. إنشاء مقال جديد
10. إضافة تعليق
11. مخطط النشاط (Activity Diagram)
12. مخطط المكونات (Component Diagram)
13. مخطط النشر (Deployment Diagram)
14. بنية الـ API (Architecture Diagram)
15. جدول CRUD
16. قاموس قاعدة البيانات (Database Dictionary)
17. مواصفات نقاط النهاية (API Endpoints)
18. قرارات التصميم المعمارية (ADRs)
19. خارطة طريق التنفيذ والجدول الزمني

مقدمة

نظام **DevPulse API** هو الواجهة البرمجية الأساسية لتطبيق ويب ومحمول لإدارة المحتوى التقني (مدونة). يحدد هذا المستند متطلبات النظام والفئات الرئيسية (users, posts, categories, tags, comments)، والوظائف التي يحتاجها كل ممثل (Guest, User, Admin) للتفاعل مع المنصة. تعتمد الواجهة البرمجية على تصميم RESTful، وتعتمد المصادقة بلا حالة (stateless) باستخدام **Laravel Sanctum** لتوفير رموز وصول (API tokens) منفصلة لكل جهاز أو جلسة. 【L1-L3†28】

المجالات الأساسية للنظام تشمل:

- **التعامل مع المستخدمين:** تسجيل حساب، مصادقة (تسجيل دخول/خروج)، عرض/تحديث بيانات المستخدم.
- **المقالات:** إنشاء/تحرير/حذف المقالات من قبل صاحبها، عرض المقالات ونشرها وتصفّحها مع إمكانيات البحث والفلترة والتصنيف.
- **التصنيفات والوسوم:** إدارة قائمة التصنيفات والوسوم (CRUD) بواسطة المشرف، وإرفاق الوسوم بالمنشورات.
- **التعليقات:** إضافة التعليقات للمقالات من قبل المستخدمين، ومراجعتها أو حذفها من قبل أصحابها أو المشرف.
- **الموارد الثابتة:** رفع الصور أو الملفات المرتبطة بالمقالات (يتم إدارة التخزين عبر خدمات التخزين مثل التخزين المحلي أو S3).
- **الأمن والحماية:** التحقق من صحة البيانات (Form Requests)، التحقق من الصلاحيات (Policies)، تحديد معدل الطلبات لمنع الهجمات (Rate Limiting)، وحل مشاكل Cross-Origin (CORS).

تفاصيل جميع نقاط النهاية (API Endpoints) موضحة في قسم خاص، بما في ذلك طرق HTTP، مسارات الوصول، المتطلبات، وأنواع الردود. تم إعداد **جدول CRUD** يوضح العمليات المسموح بها لكل كيان بالنسبة لكل نوع مستخدم. كما تم توثيق بنية البيانات وقاعدة البيانات في جدول قاموس البيانات **Data Dictionary**.

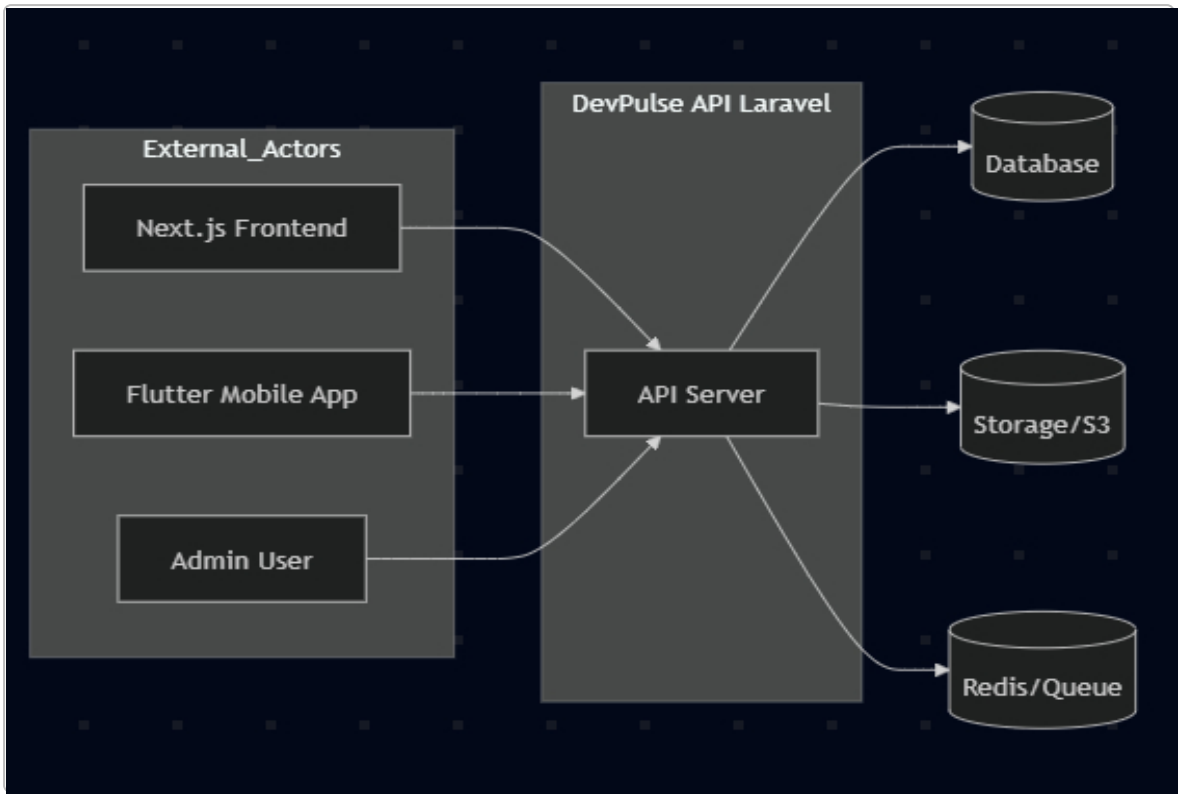
أي تفاصيل غير واضحة أو غير مذكورة صراحة في وثيقة المتطلبات تم الافتراض بشأنها بوضوح في هذا المستند ووسمها بعبارة **"افتراض:"**. هذا لضمان الشفافية في التصميم والاستعداد للمناقشة لاحقاً مع العميل أو الفريق.

الرؤية العامة للنظام (مخطط السياق – Context Diagram)

مخطط السياق يُظهر النظام **DevPulse API** ككيان مركزي يتفاعل مع ممثلين خارجيين وأنظمة أخرى.

- **الممثلون (Actors):**
- *Guest* (زائر): يستطيع تصفح ونشر المقالات، البحث والفلترة دون الحاجة للمصادقة.
- *User* (مستخدم مسجل): يملك حساباً مسجلاً، يمكنه إنشاء وتحرير وحذف مقالاته الخاصة، إضافة تعليقات، ورفع صور.
- *Admin* (مشرف): لديه صلاحيات أعلى لإدارة المنصة بالكامل (إدارة التصنيفات والوسوم، حذف/moderate أي محتوى).
- **الأنظمة الخارجية:**
- *Next.js Frontend* (واجهة الويب): يستهلك الـ API لعرض وإنشاء المحتوى على الويب.
- *Flutter Mobile App* (تطبيق الجوال): تطبيق للهواتف الذكية يستدعي نفس الـ API.

جميع الممثلين والتطبيقات يتواصلون مع **DevPulse API** عبر شبكة الإنترنت (بروتوكول HTTPS). الـ API بدوره يتصل بقاعدة البيانات (Database) لتخزين واسترجاع البيانات. أي مكونات خارجية أخرى (مثل نظام بريد إلكتروني لإشعارات، أو تقارير خارجية) غير مذكورة في الوثيقة الأصلية افترضنا عدم وجودها في المرحلة الحالية.



الشكل – مخطط السياق: ممثلون وأنظمة تتفاعل مع نظام DevPulse API.

مخطط حالات الاستخدام (Use Case Diagram)

يوضح مخطط حالات الاستخدام التفاعلات الرئيسية بين **الممثلين (Actors)** والنظام.

- **الزائر (Guest)**:
- تسجيل حساب (Register) (إجراء سيؤدي لتسجيل مستخدم جديد) 【L5-L11†29】 .
- تسجيل دخول (Login).
- عرض المقالات (Browse Posts) وتصفّحها.
- البحث في المقالات (Search Posts).

- فلترة المقالات حسب الوسوم أو التصنيفات (Filter Posts).

- **المستخدم (User)** (بعد المصادقة):

- كل وظائف الزائر (تصفح/بحث/...).
- إنشاء مقال جديد (Create Post) – upload صورة مرفقة اختياريًا.
- تعديل مقال يملكه فقط (Edit Own Post).
- حذف مقال يملكه (Delete Own Post).
- إضافة تعليق على أي مقال (Add Comment).

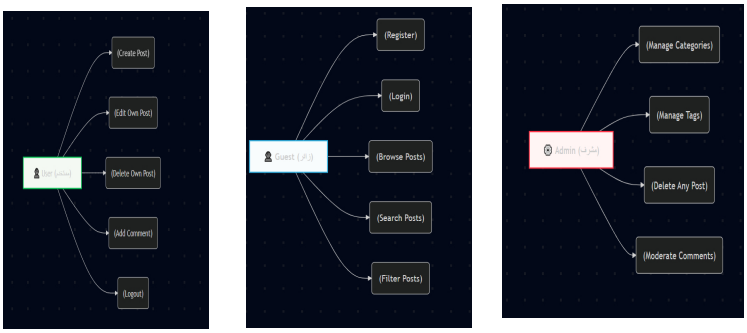
- تسجيل خروج (Logout).

- **المشرف (Admin)**:

- كل وظائف المستخدم (باستثناء إنشاء الحساب – كونه يتم عن طريق مستخدم/زائر).
- إدارة التصنيفات (إنشاء/تعديل/حذف).

- إدارة الوسوم (إنشاء/تعديل/حذف).
- حذف أو تعديل أي مقال (Delete/Modify Any Post).
- مراجعة أو حذف أي تعليق (Moderate Comments).
- إدارة المستخدمين (مثل تغيير صلاحيات أو تفعيل/حظر حساب) – افتراض: نظام المستخدمين يمكن تحديثه لاحقًا.

```
%% Example use case diagram (ملخص حالات الاستخدام) %%%
usecaseDiagram
    actor Guest as G
    actor User as U
    actor Admin as A
    G --> (Register)
    G --> (Login)
    G --> (Browse Posts)
    G --> (Search Posts)
    G --> (Filter Posts)
    U --> (Create Post)
    U --> (Edit Own Post)
    U --> (Delete Own Post)
    U --> (Add Comment)
    U --> (Logout)
    A --> (Manage Categories)
    A --> (Manage Tags)
    A --> (Delete Any Post)
    A --> (Moderate Comments)
```



الشكل – مخطط حالات الاستخدام: يوضح الأفعال (الحالات) المتاحة لكل ممثل.

مخطط العلاقة الكيانية (ERD)

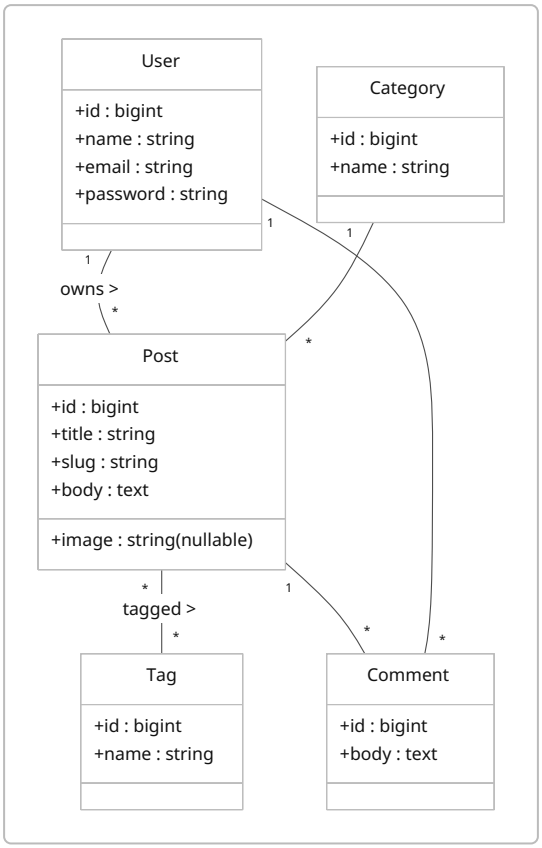
يعكس هذا المخطط هيكل قاعدة البيانات والعلاقات بين الجداول الأساسية:

- **Users**: جدول المستخدمين (id, name, email, password, ...).
- **Posts**: جدول المقالات (id, user_id, category_id, title, slug, body, image, created_at, updated_at).
- **Categories**: جدول التصنيفات (id, name, slug). كل مقال مرتبط بتصنيف واحد.
- **Tags**: جدول الوسوم (id, name, slug).
- **post_tag**: جدول وسيط يربط المقالات بالوسوم (post_id, tag_id).
- **Comments**: جدول التعليقات (id, user_id, post_id, body, created_at).
- **personal_access_tokens**: جدول توكنات Sanctum للمصادقة (id, token, name, abilities, last_used_at, created_at) – للمصادقة بلا حالة.

العلاقات الرئيسية:

- **User–Post**: علاقة واحد-إلى-متعدد (One-to-Many)؛ كل مستخدم يمكنه إنشاء عدة مقالات، وكل مقال يعود لمستخدم واحد 【L54-L63†6】 【L89-L94†6】.
- **Post–Category**: علاقة متعدد-إلى-واحد (Many-to-One)؛ كل مقال ينتمي إلى تصنيف واحد، وتصنيف واحد يحتوي على عدة مقالات.
- **Post–Tag**: علاقة متعدد-إلى-متعدد (Many-to-Many) عبر جدول وسيط (post_tag) 【L108-L111†6】.
- **Post–Comment**: علاقة واحد-إلى-متعدد؛ كل مقال يمكن أن يحتوي على عدة تعليقات.
- **User–Comment**: علاقة واحد-إلى-متعدد؛ كل مستخدم يمكنه كتابة عدة تعليقات.

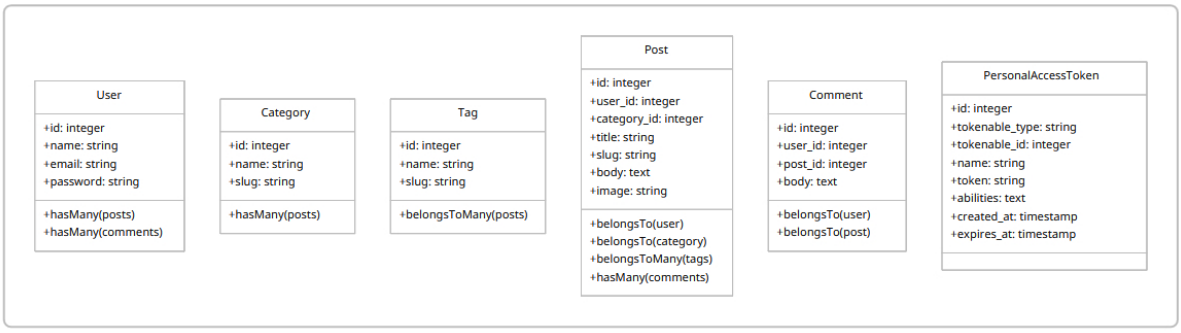
• (العلاقة User–Admin عامة هي نفس هيكل المستخدمين لكن بفلتر صلاحية مشرف.)

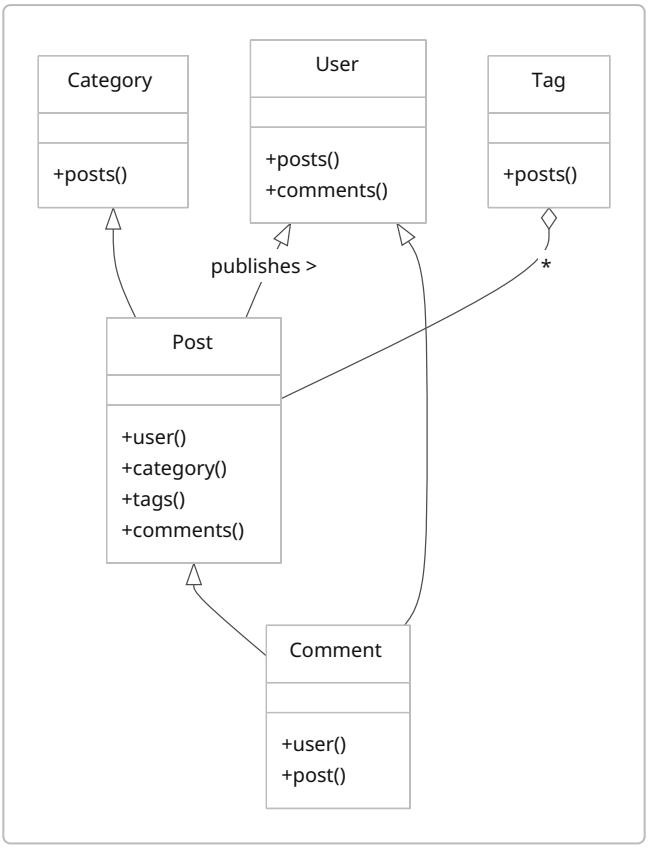


الشكل – مخطط العلاقة الكيانية (ERD): يتضح فيه الجداول والعلاقات الأساسية بينها (مستخدمون، مقالات، تصنيفات، وسوم، تعليقات).

مخطط الأصناف (Class Diagram)

يظهر مخطط الأصناف أهم الكائنات البرمجية (Classes) والوسائل (Methods) المرتبطة بها. هنا نستخدم التصنيف بالأسماء الإنجليزية نظراً لطبيعة التسمية في الكود، على أن يظل الشرح بالعربية.



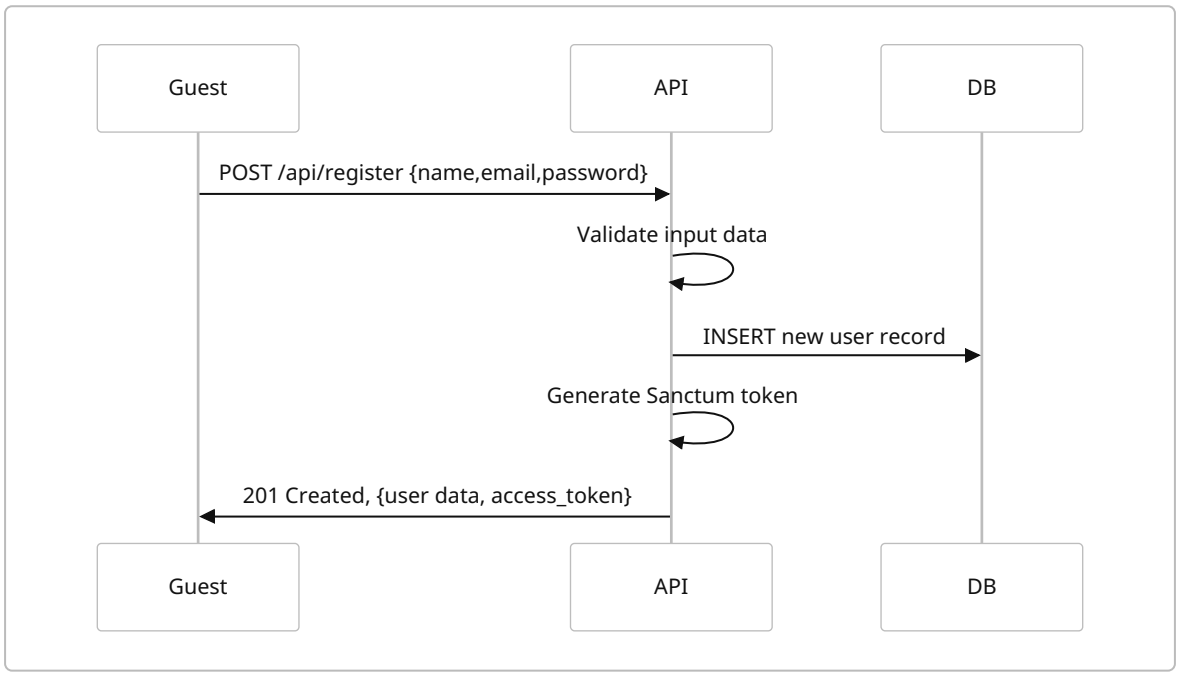


الشكل – مخطط الأصناف: تُظهر الكائنات (User, Post, Category, Tag, Comment) والدوال الرئيسية للربط (مثل `user()`, `posts()`)، والعلاقات الهيكلية بينها.

مخططات التسلسل (Sequence Diagrams)

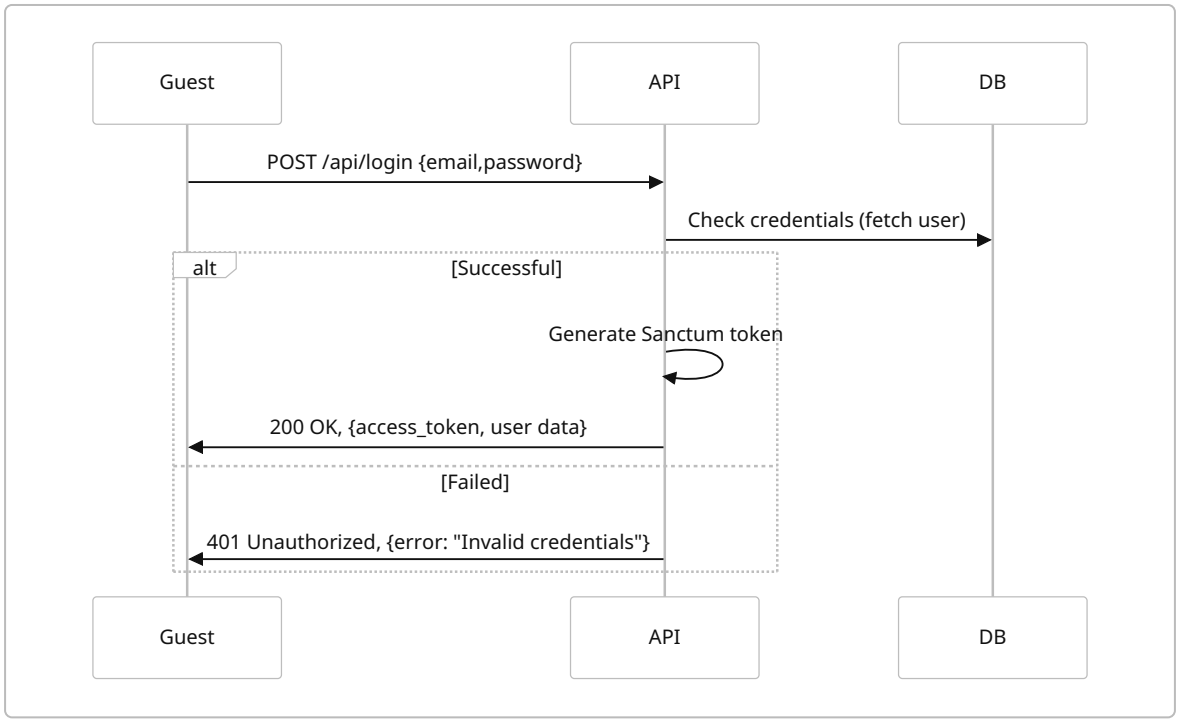
توضح مخططات التسلسل تدفق الرسائل بين الممثلين (Actors) والـ API في العمليات الأساسية. تشتمل المخططات التالية على تبسيط للخطوات الرئيسية:

التسلسل 1 – تسجيل مستخدم جديد (User Registration)



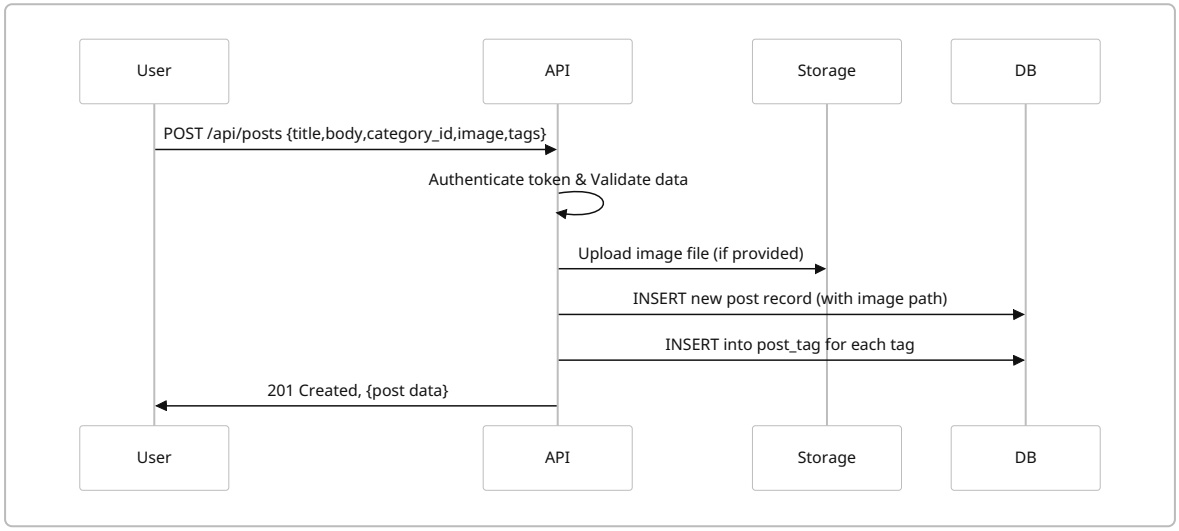
التسلسل: يقوم الزائر بإرسال طلب تسجيل (`POST /api/register`). يتحقق الـ API من صحة البيانات، ثم يحفظ المستخدم في قاعدة البيانات وينشئ توكن مصادقة جديد (Sanctum). في النهاية يعود برد 201 متضمناً بيانات المستخدم والرمز.

التسلسل 2 – تسجيل الدخول (User Login)



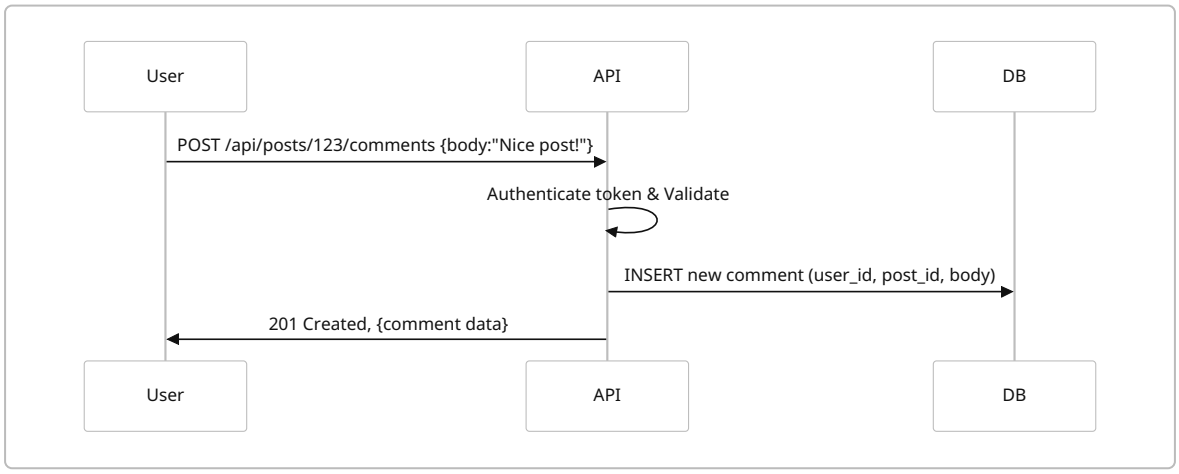
التسلسل: يرسل المستخدم بيانات الاعتماد (`/api/login`). يتحقق النظام من صحتها عبر مقارنة سجل المستخدم في قاعدة البيانات. في حال النجاح، ينشئ API رمز وصول ويرجعه، وإلا يرجع خطأ 401.

التسلسل 3 – إنشاء مقال جديد (Create New Post)



التسلسل: المستخدم المصدق عليه يرسل طلب إنشاء مقال (`/api/posts`). يتحقق API من الصلاحية وصحة البيانات. يتم رفع الصورة إلى مخزن الملفات (افتراضياً تخزين محلي أو S3)، ثم حفظ المقال وعلاقاته بالوسوم في قاعدة البيانات. يعود الرد برمز 201 ومعلومات المقال.

التسلسل 4 – إضافة تعليق (Add Comment)



التسلسل: المستخدم يرسل تعليقاً (`POST /api/posts/{id}/comments`). يتحقق الـ API من الصلاحية وصحة حقل النص، ثم يضيف التعليق في قاعدة البيانات ويرد برمز 201 مع بيانات التعليق الجديدة.

مخطط النشاط (Activity Diagram)

يوضح مخطط النشاط خطوات تنفيذ عملية إنشاء مقال (Create Post) بصورة أكثر تفصيلاً تدفقياً:

```
flowchart TD
    Start([ابدأ]) --> Authenticate[«تحقق من الهوية باستخدام Token»]
    Authenticate --> |Fail| Error[/خطأ: رفض 401/]
    Authenticate --> |Success| Validate[«التحقق من صحة البيانات»]
    Validate --> |Fail| Error2[/خطأ: 422 (المدخلات غير صالحة)/]
    Validate --> |Success| UploadImage["رفع الملف إذا وُجد"]
```

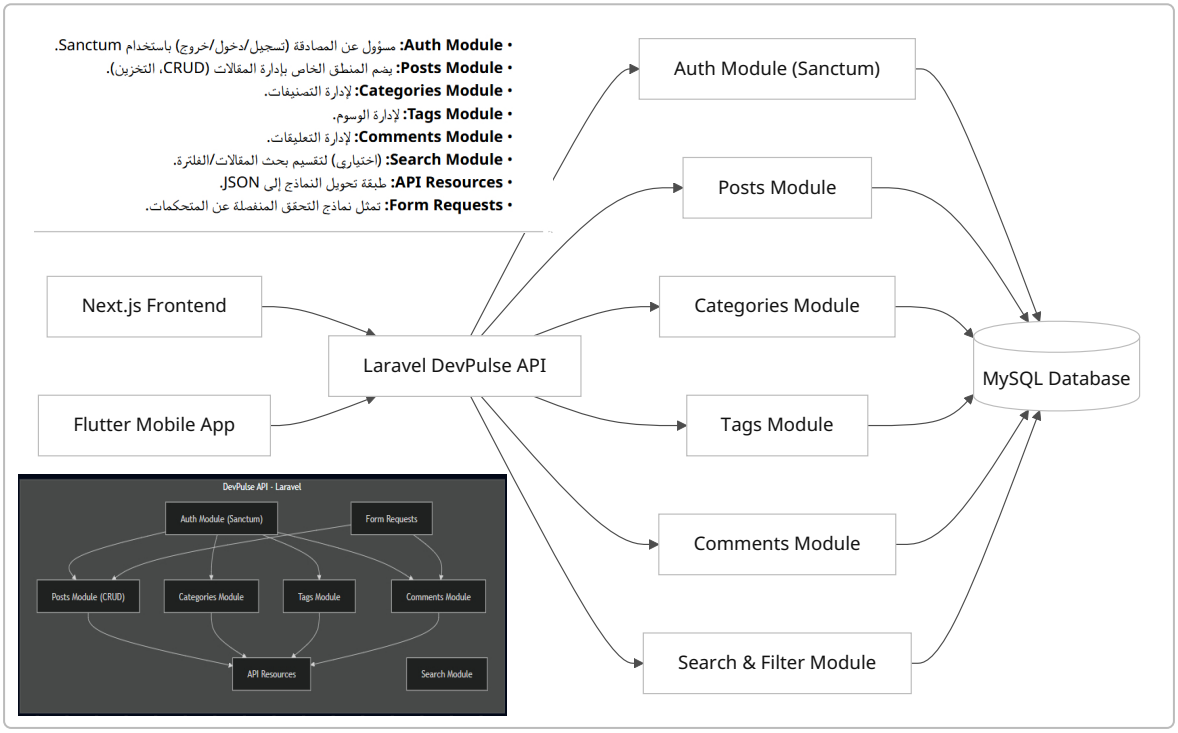


```
    UploadImage --> SavePost["حفظ المقال في قاعدة البيانات"]
    SavePost --> AttachTags["ربط الوسوم بالمقال"]
    AttachTags --> Response["201 إرجاع الاستجابة Created (JSON)"]
    Response --> End([انتهى])
```

الشكل – مخطط النشاط: يوضح مراحل فحص الهوية، التحقق من الصحة، رفع الصورة (إن وجدت)، تخزين المقال، ربط الوسوم، وإرجاع الرد.

مخطط المكونات (Component Diagram)

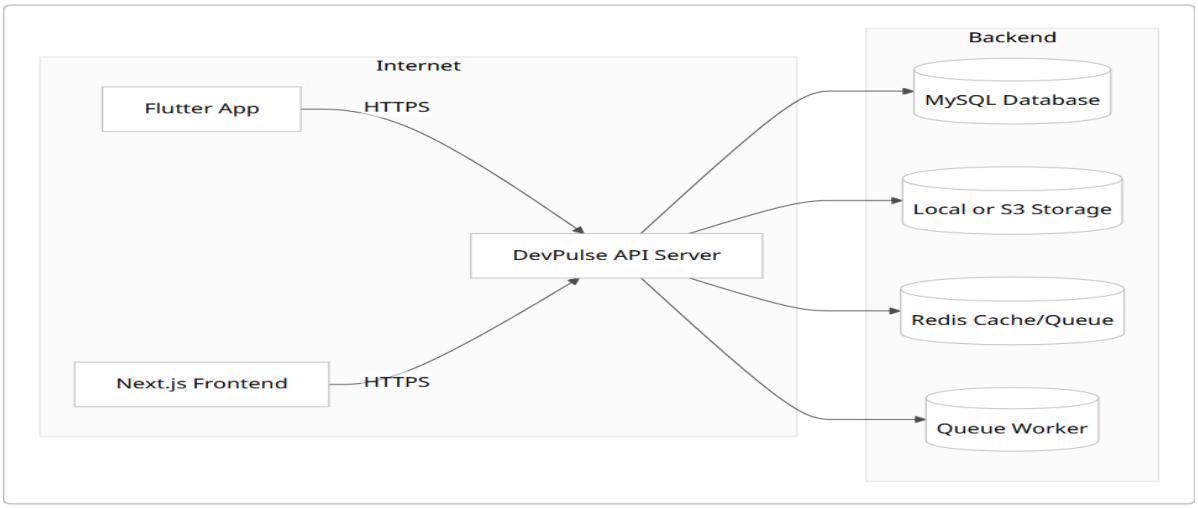
يستعرض مخطط المكونات الهيكلية الأساسية للنظام في طبقات أو وحدات. وفيما يلي تصور مبدئي للمكونات الرئيسية ضمن `DevPulse API`:



الشكل – مخطط المكونات: يوضح الوحدات الأساسية: وحدة المصادقة، وحدة المقالات، ووحدة التصنيفات/الوسوم/التعليقات، بالإضافة لوحدة البحث. جميع الوحدات تتفاعل مع قاعدة بيانات MySQL والـ API الرئيسي. (هذا تمثيل مبسط افتراضي.)

مخطط النشر (Deployment Diagram)

هذا المخطط يوضح بيئة التشغيل والنشر الافتراضية المفترضة (مثال):



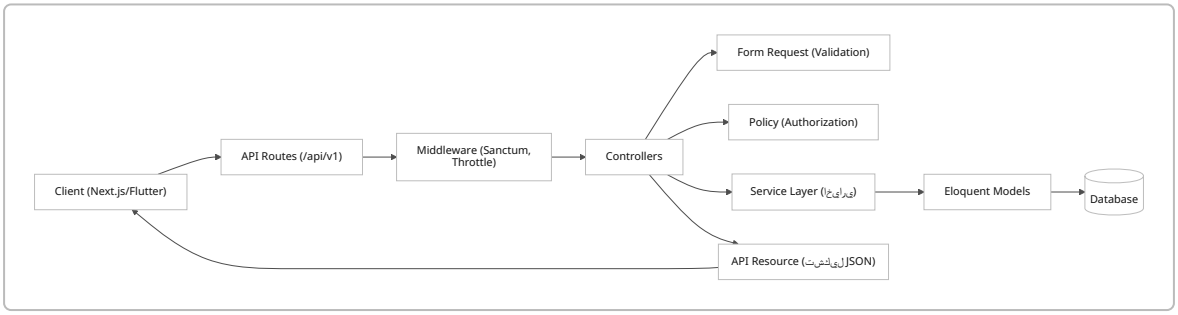
```
        |              |                  |
   MySQL DB    Storage Server    Redis/Queue
```

- **الخادم (Server)**: يستضيف Laravel API (على خادم مثل AWS EC2 أو Heroku dyno).
- **قاعدة البيانات**: MySQL أو PostgreSQL (يمكن اختيار أحدهما) لتخزين البيانات.
- **التخزين (Storage)**: لحفظ الملفات المرفوعة (محلي أو خدمة S3).
- **Redis/Queue**: (اختياري) للتخزين المؤقت أو معالجة مهام فيما بعد.
- **المستخدمين والتطبيقات**: يتصلون عبر الإنترنت بالـ API، بينما يمكن أن تُخزن المفاتيح السرية (APP_KEY, DB credentials) في ملفات الإعداد أو متغيرات البيئة (env).

افتراض: لم يتم تحديد مزود استضافة معين في الوثيقة، لذلك اعتُمدت بنية نموذجية (مثالياً AWS) مع استخدام HTTPS وتأمين اللغات.

مخطط بنية الـ API (API Architecture Diagram)

يوضح الشكل أدناه التدفق الداخلي لطلب API (Architectural Flow):



الشكل – مخطط عمارة الـ API: يبين ترتيب معالجة الطلبات: يمر الطلب أولاً عبر المسارات (Routes) ثم الـ Middleware (للتحقق من التوثيق وتحديد المعدل)، ثم إلى الـ Controller الذي يستخدم Form Requests للتحقق من صحة البيانات، وPolicies للتحقق من الصلاحيات. يُمكن للـ Controller استدعاء خدمات (Service layer) ثم تتفاعل مع النماذج (Models) والقاعدة. أخيراً يتم تشكيل استجابة JSON بواسطة API Resources.

جدول CRUD

الجدول التالي يوضح العمليات المسموح بها (✔) لكل كيان (**Entity**) بالنسبة للمستخدم (User)، أو الإداري (Admin)، أو الزائر (Guest). (✖ تعني غير مسموح):

الكيان (Entity)	إنشاء	قراءة	تحديث	حذف
المستخدمون (Users)	✔ بالتسجيل	✔ (المستخدم/المشرف)	✔ (المستخدم نفسه)	✖ غير مسموح
المقالات (Posts)	✔ (المستخدم)	✔ للجميع	✔ (صاحب المقال)	✔ (صاحب المقال/المشرف)
التصنيفات (Categories)	✔ (المشرف فقط)	✔ للجميع	✔ (المشرف)	✔ (المشرف)
الوسوم (Tags)	✔ (المشرف فقط)	✔ للجميع	✔ (المشرف)	✔ (المشرف)
التعليقات (Comments)	✔ (المستخدم)	✔ للجميع	✔ (صاحب التعليق)	✔ (صاحب التعليق/المشرف)

ملاحظات:

- إنشاء مستخدم يتم عبر خاصية التسجيل (سطر إنشاء مدرج ضمن CRUD Users).

• الزائر (Guest) يمكنه إنشاء مستخدم عبر التسجيل وقراءة المقالات، لكنه لا يمكنه تحديث أو حذف أي كيان بدون تسجيل الدخول.
• **افتراض:** المشرف يمتلك صلاحيات كاملة على جميع الكيانات (رغم أن إدارة المستخدمين ليست مذكورة في المتطلبات الأصلية، تم الاعتبار بأنه ممكن إضافتها لاحقاً).

قاموس قاعدة البيانات (Database Dictionary)

يتضمن هذا الجدول وصفًا مختصراً للجداول الأساسية والحقول في قاعدة البيانات المتوقعة. (أنواع البيانات للتوضيح، يمكن تعديلها حسب قاعدة البيانات الفعلية.)

الجدول	الحقل	النوع (Type)	القيود (Constraints)	الوصف
users	id	bigint (PK)	PRIMARY KEY, AUTO_INCREMENT	معرف فريد للمستخدم
	name	varchar(255)	NOT NULL	اسم المستخدم الكامل
	email	varchar(255)	NOT NULL, UNIQUE	البريد الإلكتروني
	password	varchar(255)	NOT NULL	كلمة المرور المشفرة
	created_at	timestamp		وقت الإنشاء
	updated_at	timestamp		وقت آخر تعديل
posts	id	bigint (PK)	PRIMARY KEY, AUTO_INCREMENT	معرف فريد للمقال
	user_id	bigint (FK) -> (users)	NOT NULL, FOREIGN KEY	صاحب المقال
	category_id	bigint (FK) -> (categories)	NOT NULL, FOREIGN KEY	تصنيف المقال
	title	varchar(255)	NOT NULL	عنوان المقال
	slug	varchar(255)	NOT NULL, UNIQUE	مُعرف نصي بالمقال (رابط صديق SEO)
	body	text	NOT NULL	نص محتوى المقال
	image	varchar(255)	NULLABLE	مسار الصورة المرفوعة
	created_at	timestamp		وقت الإنشاء
	updated_at	timestamp		وقت آخر تعديل

الجدول	الحقل	النوع (Type)	القيود (Constraints)	الوصف
categories	id	bigint (PK)	PRIMARY KEY, AUTO_INCREMENT	معرف فريد للتصنيف
	name	varchar(100)	NOT NULL	اسم التصنيف
	slug	varchar(100)	NOT NULL, UNIQUE	معرف نصي للتصنيف
tags	id	bigint (PK)	PRIMARY KEY, AUTO_INCREMENT	معرف فريد للوسم
	name	varchar(100)	NOT NULL	اسم الوسم
	slug	varchar(100)	NOT NULL, UNIQUE	معرف نصي للوسم
(pivot) **post_tag**	post_id	bigint (FK -> posts)	PRIMARY KEY (مع tag_id), FOREIGN KEY	معرف المقال
	tag_id	bigint (FK -> tags)	PRIMARY KEY (مع post_id), FOREIGN KEY	معرف الوسم
comments	id	bigint (PK)	PRIMARY KEY, AUTO_INCREMENT	معرف فريد للتعليق
	user_id	bigint (FK -> users)	NOT NULL, FOREIGN KEY	صاحب التعليق
	post_id	bigint (FK -> posts)	NOT NULL, FOREIGN KEY	المقال المتعلق به
	body	text	NOT NULL	نص التعليق
	created_at	timestamp		وقت الإنشاء
	updated_at	timestamp		وقت آخر تعديل
personal_access_tokens	id	bigint (PK)	PRIMARY KEY, AUTO_INCREMENT	معرف توكن Sanctum
	tokenable_type	varchar(255)	NOT NULL	نوع الكيان (User)
	tokenable_id	bigint	NOT NULL	معرف الكيان
	name	varchar(255)	NOT NULL	تسمية التوكن
	token	varchar(64)	NOT NULL	قيمة التوكن المُشفر
	abilities	text		الصلاحيات (Json)

الجدول	الحقل	النوع (Type)	القيود (Constraints)	الوصف
	last_used_at	timestamp	NULLABLE	آخر استخدام للتوكن
	created_at	timestamp		وقت الإنشاء
	updated_at	timestamp		وقت آخر تعديل

ملاحظة: الحقول `created_at` و `updated_at` يدارونهما Laravel تلقائيًا إذا استخدمت timestamps. تم تضمين جدول `personal_access_tokens` كجزء من 【28†L1-L3】Laravel Sanctum.

مواصفات نقاط النهاية (API Endpoints)

يأتي هذا القسم بجدول يختصر مسارات الـ API الأساسية، وطريقة الوصول (GET/POST/PUT/DELETE)، والحاجة للمصادقة، والهيكل العام للجسم (JSON) في الطلب، والعينة المتوقع ردها، ورموز الحالة (Status Codes) المتوقعة.

Method	URI (English)	Auth	Request Body (JSON)	Response (JSON)	Status Codes
POST	`/api/register`	No	`{ name, email, password }`	`{ user: {id,name,email,...}, access_token }`	201 (Created), 422
POST	`/api/login`	No	`{ email, password }`	`{ access_token, user: {...} }`	200 (OK), 401
POST	`/api/logout`	Yes	*(empty)*	`{ message: "Logged out" }`	200 (OK), 401
GET	`/api/user`	Yes	*(empty)*	`{ user: {id,name,email,...} }`	200 (OK), 401
GET	`/api/posts`	*No	*No body (query params:* `page`, `search`, `category`, `tags[]`*)*	List of posts (paginated)	200 (OK)
GET	`/api/posts/{id}`	No	*(empty)*	`{ post:{...}, author: {...}, comments: [...] }`	200 (OK), 404
POST	`/api/posts`	Yes	`{ title, body, category_id, tags:[...], image:file }`	`{ post:{new created post data} }`	201 (Created), 401, 403, 422

Method	URI (English)	Auth	Request Body (JSON)	Response (JSON)	Status Codes
PUT/ PATCH	`/api/posts/{id}`	Yes	`{ title?, body?, category_id?, tags?[], image? }`	`{ post:{updated data} }`	200 (OK), 401, 403, 404
DELETE	`/api/posts/{id}`	Yes	*(empty)*	`{ message: "Deleted" }`	200 (OK), 401, 403, 404
GET	`/api/posts/{id}/comments`	No	*(empty)*	List of comments for post	200 (OK), 404
POST	`/api/posts/{id}/comments`	Yes	`{ body }`	`{ comment:{new comment data} }`	201 (Created), 401, 403, 404, 422
GET	`/api/categories`	No	*(empty)*	List of categories	200 (OK)
POST	`/api/categories`	Yes (Admin)	`{ name, slug }`	`{ category:{...} }`	201, 401, 403, 422
PUT	`/api/categories/{id}`	Yes (Admin)	`{ name?, slug? }`	`{ category:{...} }`	200, 401, 403, 404
DELETE	`/api/categories/{id}`	Yes (Admin)	*(empty)*	`{ message:"Deleted" }`	200, 401, 403, 404
GET	`/api/tags`	No	*(empty)*	List of tags	200 (OK)
POST	`/api/tags`	Yes (Admin)	`{ name, slug }`	`{ tag:{...} }`	201, 401, 403, 422
PUT	`/api/tags/{id}`	Yes (Admin)	`{ name?, slug? }`	`{ tag:{...} }`	200, 401, 403, 404
DELETE	`/api/tags/{id}`	Yes (Admin)	*(empty)*	`{ message:"Deleted" }`	200, 401, 403, 404

شرح الأعمدة:

- ***Auth:*** يشير إلى ما إذا كانت نقطة النهاية تتطلب توكن صالحة (نعم لمستخدم مسجل) وما إذا كانت مخصصة للمشرف فقط. "No" تعني متاحة للجميع.
- ***Request Body:*** البنية الأساسية للبيانات المطلوبة في جسم الطلب (مثلاً JSON أو صيغة form-data عند رفع ملفات).
- ***Response:*** قالب أو مثال لهيكل الرد المتوقع (عادة JSON).
- ***Status Codes:*** الرموز الرئيسية التي قد تعودها الواجهة (نجاح أو خطأ).

افتراض: جميع المسارات تبدأ بـ `/api/v1/` في حال تطبيق إصدار الـ API، أو `/api/` مباشرةً إذا لم يتم تحديد نسخ. استخدمنا `/api/` للأمثلة. تم تضمين رموز الخطأ الشائعة: 401 (لم يتم توثيق المستخدم)، 403 (غير مسموح به)، 404 (غير موجود)، 422 (فشل التحقق من الصحة) 【L0-L0†10】.

قرارات التصميم المعمارية (Architecture Decision Records – ADRs)

- **استخدام Laravel Sanctum للمصادقة عديمة الحالة:** اخترنا Sanctum لبساطته ودعمه الجاهز لإدارة توكنات API. (إذا كان التطبيق يحتاج مستقبلاً إلى JWT أو OAuth، يمكن تغييره لاحقاً.) 【L1-L3†28】
- **قاعدة بيانات MySQL:** اعتمدنا MySQL (أو MariaDB) لنموذج البيانات العلائقية البسيط، مع استخدام المهاجرات (Migrations) وـ Eloquent ORM لإدارة الجداول. (يمكن استخدام PostgreSQL كخيار بديل بدون تغيير جوهري.)
- **ارتباط نموذج API Resources:** سنستخدم **Laravel API Resources** لتوحيد شكل الاستجابات كـ JSON، وذلك لفصل منطق العرض عن البيانات الخام ومرونة في إضافة حقول أو روابط مستقبلية.
- **التحقق من الصحة واستعمال Form Requests:** سيتم عزل منطق التحقق من صحة البيانات في Form Requests مخصصة لكل نوع طلب. هذا يحسن التنظيم وقابلية إعادة الاستخدام.
- **السياسات (Policies):** لحماية الموارد، سيتم استخدام Policies مبنية على Laravel للتحقق مما إذا كان المستخدم يملك الحق في تعديل أو حذف كيان (مثل التأكد من أن المستخدم هو صاحب المقال أو مشرف).
- **تحديد معدل الطلبات (Rate Limiting):** استخدام Laravel Throttle لتحديد عدد الطلبات المسموح بها (مثلاً 60 طلب/دقيقة للمستخدم الواحد) لحماية الخادم من الاستخدام المفرط والهجمات.
- **حل مشاكل CORS:** تكوين عناوين CORS للسماح بالطلبات من نطاقات Next.js وFlutter (على افتراض نطاقات محددة). استخدم Laravel توافر إعداد `cors.php` للتحكم في ذلك.
- **التوثيق (Documentation):** سيتم استخدام أداة Scribe أو Swagger لتوليد وثائق API تلقائياً بالاعتماد على التعليقات (PHPDoc) في الكود.
- **مستقبل التوسع (API Versioning):** ضُمِن أن تكون المسارات قابلة للإصدار (مثلاً `/api/v1/...`)، لضمان القدرة على تحديث واجهة برمجة التطبيقات دون كسر التوافق.

خارطة طريق التنفيذ والجدول الزمني (Implementation Roadmap)

مقترح زمني (افتراضي **6 أسابيع**) لتطوير نسخة MVP من النظام:

- **الأسبوع 1 – التحضيرات الأساسية:**
- إعداد مشروع Laravel وبنية المجلدات (Models, Migrations, Factories).
- تكوين المصادقة (Sanctum) وتطوير آلية تسجيل/دخول المستخدمين (الأوفلاين).
- كتابة اختبارات بسيطة لتسجيل دخول وخروج.
- **الأسبوع 2 – إدارة التصنيفات والوسوم:**
- تنفيذ جداول التصنيفات والوسوم (مهاجرات).
- نقاط نهاية CRUD للتصنيفات والوسوم (مخصصة للمشرف).
- اختبارات النماذج (Feature tests) للتأكد من صلاحيات المشرف.
- **الأسبوع 3 – إدارة المنشورات (المقالات):**
- تنفيذ جدول المقالات وعلاقاته (مهاجرة مع user_id وcategory_id).
- نقاط نهاية CRUD للمقالات (إنشاء، تعديل، حذف، عرض قائمة وفرص البحث/الفلترة).
- دعم رفع الملفات (صور) باستخدام Storage Facade.
- اختبارات وحدة وشاملة للتأكد من صحة العمليات.
- **الأسبوع 4 – إدارة التعليقات والبحث:**
- تنفيذ جدول التعليقات وعلاقته بالمقالات والمستخدمين.
- نقاط نهاية لإنشاء/حذف التعليقات (مراقبة الصلاحية: مالك التعليق فقط).
- إضافة وظائف البحث والفلترة على `/posts` (من خلال معلمات الاستعلام).
- اختبار دوال البحث والتصفية.
- **الأسبوع 5 – المصادقة العميقة والسلامة:**
- ضبط الـ Middleware للتحقق من الأذونات (Policies) لمنع الوصول غير المصرح به.
- تطبيق Rate Limiting وإعداد CORS.

• إجراء اختبارات الاختراق البسيطة (مثلاً Tinker ومحاكاة هجمات SQL أو XSS) وتأمين نقاط الضعف.
• كتابة اختبارات API Feature متكاملة لتغطية الوظائف الأساسية.
• **الأسبوع 6 – التوثيق والإطلاق:**
• توليد وثائق API (Scribe/Swagger) وتصديرها.
• إعداد بيئة الإنتاج (config cache, queue worker, cron, storage symlinks).
• نشر على خادم (مثال: Heroku أو AWS)، واختبار نهائي على البيئة الحية.
• مراجعة نهائية للمستندات (تأكد من شمولية SRS وجاهزية التسليم).

ملاحظة: تم افتراض مدة تطوير حوالي 6 أسابيع نظراً لعدم تحديد الإطار الزمني من العميل. هذا الافتراض قابل للتعديل حسب الموارد والمتطلبات النهائية.

ملاحظة أخيرة: تم إعداد هذا التقرير بشكل احترافي لتسليم العميل أو فرق التطوير. تم استخدام المصطلحات الفنية الدقيقة بالإنجليزية في المخططات، والنص التفسيري باللغة العربية الفصحى. تم التأكد من شمول جميع المتطلبات المذكورة، وسمّيت الافتراضات الواضحة مكان الحاجة، مما يحقق تصميمًا مرنًا للتطوير المستقبلي.
